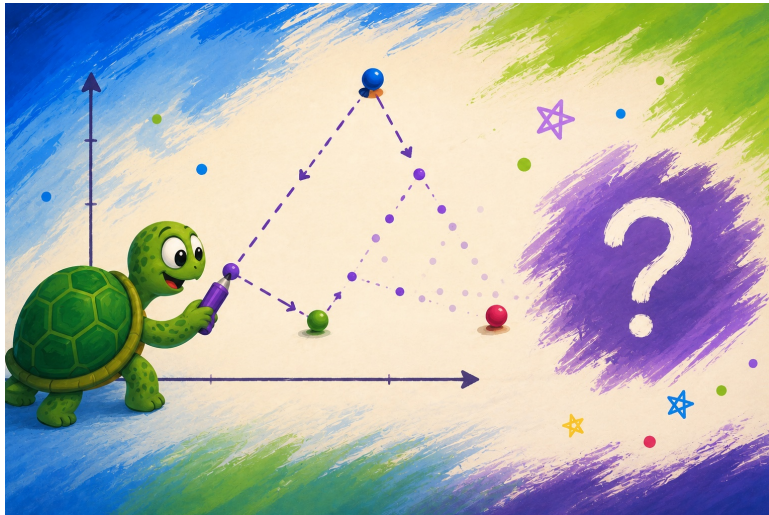


Sierpiński



This works a locally installed editor, such as Thonny or the Raspberry Pi online editor:
• editor.raspberrypi.org

This project will draw a **fractal**. A fractal is a pattern that contains smaller and smaller copies of itself. You can keep zooming in forever and still find the same pattern.

- 1** We are going to write a very small program using random choices. Can random dots really make this beautiful pattern?

This project will use the **turtle graphics** library for drawing and the `choice` method from the **random** library that chooses an item from a list at random.

Note: A **library** is a set of extra commands that you can add for a particular task. This is the same as the extensions in Scratch. The **turtle graphics** library in Python is like the **pen** extension in Scratch.

- 2** Open a new Python program and add the lines on the right.

- The `import turtle` command says we are going to use all of the turtle graphics commands.

```
import turtle
from random import choice
```

- The `from random import choice` says we are only going to use the `choice` command from the `random` library.

In Python you can add the whole of a library, like `turtle graphics`, or just pick the commands you want, like `choice` from the `random` library.

- 3** First of all,

- Choose a point on the screen. Later this will be one corner of the triangle.
- Then turn the turtle towards that point and move towards it.

```
import turtle
from random import choice

point = (100, 100)
angle = turtle.towards(point)
print("The angle to the point is", angle, "degrees")

print("Turning the turtle towards that point ...")
turtle.setheading(angle)

print("Drawing a line in that direction ...")
turtle.forward(100)
```

Note: The `print` commands are just there to show you what is happening. They will be removed in later code examples, and you can remove them too.

4

We don't want to draw lines, and to draw the Sierpiński triangle we will want to move forward half the distance to the point, not 100 every time.

- Use penup to stop the line being drawn.
- Find half the distance from the turtle to the point, and move forward that amount.
- Draw a dot when the turtle arrives.
- Do this 10 times and see the dotted pattern you get.

```
import turtle
from random import choice

turtle.penup()

point = (100, 100)
for i in range(10):
    angle = turtle.towards(point)
    turtle.setheading(angle)

    distance = turtle.distance(point) / 2
    print("Half the distance to the point is", distance)

    turtle.forward(distance)
    print("Drawing a new dot ...")
    turtle.dot(5)
```

This cuts the distance from the turtle to the point in half. Why a half? Actually, this is the secret that makes the famous pattern!

Notice that these two lines

```
angle = turtle.towards(point)
turtle.setheading(angle)
```

are not new lines but they are now indented with spaces at the start. This is important to make sure they are in the for loop.

5

The final step is to have three corners, and for each dot we will pick a corner to travel toward at random.

- Create a list of three points.
- Each time round the for loop pick a random corner with the choice command. This comes from the `random` library.
- Make the turtle super fast by setting the speed to 11.
- Draw 1000 dots instead of 10.

```
import turtle
from random import choice

turtle.penup()
turtle.speed(11)

corners = [
    (-200, -200),
    (0, 100),
    (200, -200)
]

for i in range(1000):
    point = choice(corners)

    angle = turtle.towards(point)
    turtle.setheading(angle)

    distance = turtle.distance(point) / 2

    turtle.forward(distance)
    turtle.dot(5)
```

Notice, nobody tells the turtle where to draw the triangle. It only follows one simple rule: pick a corner at random and move half-way towards it. Somehow, that simple rule creates one of the most famous fractals in mathematics. This is an example of how complicated patterns can grow from very simple rules.

6

Some things to try:

- 1000 dots may still not be enough. Try more, or even go on forever.
- The dots can be a bit blobby. Can you work out how to make the dots smaller so that the pattern is clearer?
- Try different corners to make different triangles.
- **(Tricky)** What happens if you have a different colour for when you go towards each corner? There are different ways to do this!
- **(Very tricky!)** You can have more than three corners but the 'secret' halving of the distance needs to be changed to something else.